

18.6 Handling Internal Boundary Conditions or Singular Points

Singularities can occur in the interiors of two-point boundary value problems. Typically, there is a point x_s at which a derivative must be evaluated by an expression of the form

$$S(x_s) = \frac{N(x_s, \mathbf{y})}{D(x_s, \mathbf{y})} \quad (18.6.1)$$

where the denominator $D(x_s, \mathbf{y}) = 0$. In physical problems with finite answers, singular points usually come with their own cure: Where $D \rightarrow 0$, there the physical solution $\mathbf{y}$ must be such as to make $N \rightarrow 0$ simultaneously, in such a way that the ratio takes on a meaningful value. This constraint on the solution $\mathbf{y}$ is often called a *regularity condition*. The condition that $D(x_s, \mathbf{y})$ satisfy some special constraint at x_s is entirely analogous to an extra boundary condition, an algebraic relation among the dependent variables that must hold at a point.

We discussed a related situation earlier, in §18.2, when we described the “fitting point method” to handle the task of integrating equations with singular behavior at the boundaries. In those problems you are unable to integrate from one side of the domain to the other. However, the ODEs do have well-behaved derivatives and solutions in the neighborhood of the singularity, so it is readily possible to integrate away from the point. Both the relaxation method and the method of shooting to a fitting point handle such problems easily. Also, in those problems the presence of singular behavior served to isolate some special boundary values that had to be satisfied to solve the equations.

The difference here is that we are concerned with singularities arising at intermediate points, where the location of the singular point depends on the solution, so is not known a priori. Consequently, we face a circular task: The singularity prevents us from finding a numerical solution, but we need a numerical solution to find its location. Such singularities are also associated with selecting a special value for some variable that allows the solution to satisfy the regularity condition at the singular point. Thus, internal singularities take on aspects of being internal boundary conditions.

One way of handling internal singularities is to treat the problem as a free boundary problem, as discussed at the end of §18.0. Suppose, as a simple example, we consider the equation

$$\frac{dy}{dx} = \frac{N(x, y)}{D(x, y)} \quad (18.6.2)$$

where N and D are required to pass through zero at some unknown point x_s . We add the equation

$$z \equiv x_s - x_1 \quad \frac{dz}{dx} = 0 \quad (18.6.3)$$

where x_s is the unknown location of the singularity, and change the independent variable to t by setting

$$x - x_1 = tz, \quad 0 \leq t \leq 1 \quad (18.6.4)$$

The boundary conditions at $t = 1$ become

$$N(x, y) = 0, \quad D(x, y) = 0 \quad (18.6.5)$$

Use of an adaptive mesh as discussed in the previous section is another way to overcome the difficulties of an internal singularity. For the problem (18.6.2), we add the mesh spacing equations

$$\frac{dQ}{dq} = \psi \quad (18.6.6)$$

$$\frac{d\psi}{dq} = 0 \quad (18.6.7)$$

with a simple mesh spacing function that maps x uniformly into q , where q runs from 0 to $M - 1$, with M the number of mesh points:

$$Q(x) = x - x_1, \quad \frac{dQ}{dx} = 1 \quad (18.6.8)$$

Having added three first-order differential equations, we must also add their corresponding boundary conditions. If there were no singularity, these could simply be

$$\text{at } q = 0 : \quad x = x_1, \quad Q = 0 \quad (18.6.9)$$

$$\text{at } q = M - 1 : \quad x = x_2 \quad (18.6.10)$$

and a total of N values y_i specified at $q = 0$. In this case the problem is essentially an initial value problem with all boundary conditions specified at x_1 and the mesh spacing function is superfluous.

However, in the actual case at hand we impose the conditions

$$\text{at } q = 0 : \quad x = x_1, \quad Q = 0 \quad (18.6.11)$$

$$\text{at } q = M - 1 : \quad N(x, y) = 0, \quad D(x, y) = 0 \quad (18.6.12)$$

and $N - 1$ values y_i at $q = 0$. The “missing” y_i is to be adjusted, in other words, so as to make the solution go through the singular point in a regular (zero-over-zero) rather than irregular (finite-over-zero) manner. Notice also that these boundary conditions do not directly impose a value for x_2 , which becomes an adjustable parameter that the code varies in an attempt to match the regularity condition.

In this example the singularity occurred at a boundary, and the complication arose because the location of the boundary was unknown. In other problems we might wish to continue the integration beyond the internal singularity. For the example given above, we could simply integrate the ODEs to the singular point, and then as a separate problem recommence the integration from the singular point on as far we care to go. However, in other cases the singularity occurs internally, but does not completely determine the problem: There are still some more boundary conditions to be satisfied further along in the mesh. Such cases present no difficulty in principle, but do require some adaptation of the relaxation code given in §18.3. In effect, all you need to do is to add a “special” block of equations at the mesh point where the internal boundary conditions occur, and do the proper bookkeeping.

Figure 18.6.1 illustrates a concrete example where the overall problem contains five equations with two boundary conditions at the first point, one “internal” boundary condition, and two final boundary conditions. The figure shows the structure of the overall matrix equations along the diagonal in the vicinity of the special block. In the middle of the domain, blocks typically involve five equations (rows) in ten unknowns (columns). For each block prior to the special block, the initial boundary conditions provided enough information to zero the first two columns of the blocks. The five FDEs eliminate five more columns, and the final three columns need to be stored for the backsubstitution step (as described in §18.3). To handle the extra condition, we break the normal cycle and add a special block with only one equation: the internal boundary condition. This effectively reduces the required storage of unreduced coefficients by one column for the rest of the grid, and allows us to reduce to zero the first three columns of subsequent blocks. The functions `red`, `pinvs`, and `bksub` can readily handle these cases with minor recoding, but each problem makes for a special case, and you will have to make the modifications as required.

CITED REFERENCES AND FURTHER READING:

London, R.A., and Flannery, B.P. 1982, “Hydrodynamics of X-Ray Induced Stellar Winds,” *Astrophysical Journal*, vol. 258, pp. 260–269.

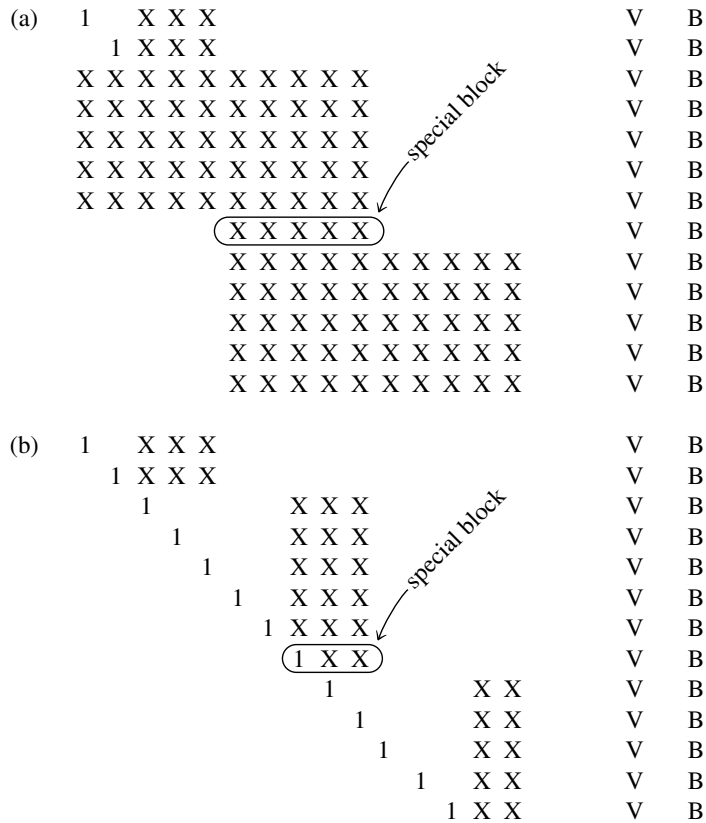


Figure 18.6.1. FDE matrix structure with an internal boundary condition. The internal condition introduces a special block. (a) Original form, compare with Figure 18.3.1; (b) final form, compare with Figure 18.3.2.

Integral Equations and Inverse Theory

19.0 Introduction

Many people, otherwise numerically knowledgeable, imagine that the numerical solution of integral equations must be an extremely arcane topic, since, until recently, it was almost never treated in numerical analysis textbooks. Actually there is a large and growing literature on the numerical solution of integral equations, including several good monographs [1-3]. One reason for the sheer volume of this activity is that there are many different kinds of equations, each with many different possible pitfalls; often many different algorithms have been proposed to deal with a single case.

There is a close correspondence between linear integral equations, which specify linear, integral relations among functions in an infinite-dimensional function space, and plain old linear equations, which specify analogous relations among vectors in a finite-dimensional vector space. Because this correspondence lies at the heart of most computational algorithms, it is worth making it explicit as we recall how integral equations are classified.

Fredholm equations involve definite integrals with fixed upper and lower limits. An *inhomogeneous Fredholm equation of the first kind* has the form

$$g(t) = \int_a^b K(t, s) f(s) ds \quad (19.0.1)$$

Here $f(t)$ is the unknown function to be solved for, while $g(t)$ is a known “right-hand side.” (In integral equations, for some odd reason, the familiar “right-hand side” is conventionally written on the left!) The function of two variables, $K(t, s)$, is called the *kernel*. Equation (19.0.1) is analogous to the matrix equation

$$\mathbf{K} \cdot \mathbf{f} = \mathbf{g} \quad (19.0.2)$$

whose solution is $\mathbf{f} = \mathbf{K}^{-1} \cdot \mathbf{g}$, where $\mathbf{K}^{-1}$ is the matrix inverse. Like equation (19.0.2), equation (19.0.1) has a unique solution whenever g is nonzero (the homogeneous case with $g = 0$ is almost never useful) and K is invertible. However, as we shall see, this latter condition is as often the exception as the rule.

The analog of the finite-dimensional eigenvalue problem

$$(\mathbf{K} - \sigma \mathbf{1}) \cdot \mathbf{f} = \mathbf{g} \quad (19.0.3)$$

is called a *Fredholm equation of the second kind*, usually written

$$f(t) = \lambda \int_a^b K(t, s) f(s) ds + g(t) \quad (19.0.4)$$

Again, the notational conventions do not exactly correspond: λ in equation (19.0.4) is $1/\sigma$ in (19.0.3), while $\mathbf{g}$ is $-\mathbf{g}/\lambda$. If g (or $\mathbf{g}$) is zero, then the equation is said to be *homogeneous*. If the kernel $K(t, s)$ is bounded, then, like equation (19.0.3), equation (19.0.4) has the property that its homogeneous form has solutions for at most a denumerably infinite set $\lambda = \lambda_n$, $n = 1, 2, \dots$, the *eigenvalues*. The corresponding solutions $f_n(t)$ are the *eigenfunctions*. The eigenvalues are real if the kernel is symmetric.

In the *inhomogeneous* case of nonzero g (or $\mathbf{g}$), equations (19.0.3) and (19.0.4) are soluble *except* when λ (or σ) is an eigenvalue — because the integral operator (or matrix) is singular then. In integral equations this dichotomy is called *the Fredholm alternative*.

Fredholm equations of the first kind are often extremely ill-conditioned. Applying the kernel to a function is generally a smoothing operation, so the solution, which requires inverting the operator, will be extremely sensitive to small changes or errors in the input. Smoothing often actually loses information, and there is no way to get it back in an inverse operation. Specialized methods have been developed for such equations, which are often called *inverse problems*. In general, a method must augment the information given with some prior knowledge of the nature of the solution. This prior knowledge is then used, in one way or another, to restore lost information. We will introduce such techniques in §19.4.

Inhomogeneous Fredholm equations of the second kind are much less often ill-conditioned. Equation (19.0.4) can be rewritten as

$$\int_a^b [K(t, s) - \sigma \delta(t - s)] f(s) ds = -\sigma g(t) \quad (19.0.5)$$

where $\delta(t - s)$ is a Dirac delta function (and where we have changed from λ to its reciprocal σ for clarity). If σ is large enough in magnitude, then equation (19.0.5) is, in effect, diagonally dominant and thus well-conditioned. Only if σ is small do we go back to the ill-conditioned case.

Homogeneous Fredholm equations of the second kind are likewise not particularly ill-posed. If K is a smoothing operator, then it will map many f 's to zero, or near-zero; there will thus be a large number of degenerate or nearly degenerate eigenvalues around $\sigma = 0$ ($\lambda \rightarrow \infty$), but this will cause no particular computational difficulties. In fact, we can now see that the magnitude of σ needed to rescue the inhomogeneous equation (19.0.5) from an ill-conditioned fate is generally much *less* than that required for diagonal dominance. Since the σ term shifts all eigenvalues, it is enough that it be large enough to shift a smoothing operator's forest of near-zero eigenvalues away from zero, so that the resulting operator becomes invertible (except, of course, at the discrete eigenvalues).

Volterra equations are a special case of Fredholm equations with $K(t, s) = 0$ for $s > t$. Chopping off the unnecessary part of the integration, Volterra equations are written in a form where the upper limit of integration is the independent variable t . The *Volterra equation of the first kind*,

$$g(t) = \int_a^t K(t, s) f(s) ds \quad (19.0.6)$$

has as its analog the matrix equation (now written out in components)

$$\sum_{j=0}^k K_{kj} f_j = g_k \quad (19.0.7)$$

Comparing with equation (19.0.2), we see that the Volterra equation corresponds to a matrix $\mathbf{K}$ that is lower (i.e., left) triangular, with zero entries above the diagonal. As we know from Chapter 2, such matrix equations are trivially soluble by forward substitution. Techniques for solving Volterra equations are similarly straightforward. When experimental measurement noise does not dominate, Volterra equations of the first kind tend *not* to be ill-conditioned; the upper limit to the integral introduces a sharp step that conveniently spoils any smoothing properties of the kernel.

The Volterra equation of the second kind is written

$$f(t) = \int_a^t K(t, s) f(s) ds + g(t) \quad (19.0.8)$$

whose matrix analog is the equation

$$(\mathbf{K} - \mathbf{1}) \cdot \mathbf{f} = \mathbf{g} \quad (19.0.9)$$

with $\mathbf{K}$ lower triangular. The reason there is no λ in these equations is that (i) in the inhomogeneous case (nonzero g) it can be absorbed into K , while (ii) in the homogeneous case ($g = 0$), it is a theorem that Volterra equations of the second kind with bounded kernels have no eigenvalues with square-integrable eigenfunctions.

We have specialized our definitions to the case of linear integral equations. In a nonlinear version of equation (19.0.1) or (19.0.6), instead of $K(t, s) f(s)$ the integrand would be $K(t, s, f(s))$. A nonlinear version of equation (19.0.4) or (19.0.8) would have an integrand $K(t, s, f(t), f(s))$. Nonlinear Fredholm equations are considerably more complicated than their linear counterparts. Fortunately, they do not occur as frequently in practice and we shall by and large ignore them in this chapter. By contrast, solving nonlinear Volterra equations usually involves only a slight modification of the algorithm for linear equations, as we shall see.

Almost all methods for solving integral equations numerically make use of *quadrature rules*, frequently Gaussian quadratures. This would be a good time for you to go back and review §4.6, especially the advanced material toward the end of that section.

In the sections that follow, we first discuss Fredholm equations of the second kind with smooth kernels (§19.1). Nontrivial quadrature rules come into the discussion, but we will be dealing with well-conditioned systems of equations. We then return to Volterra equations (§19.2), and find that simple and straightforward methods are generally satisfactory for these equations.

In §19.3 we discuss how to proceed in the case of singular kernels, focusing largely on Fredholm equations (both first and second kinds). Singularities require special quadrature rules, but they are also sometimes blessings in disguise, since they can spoil a kernel's smoothing and make problems well-conditioned.

In §19.4 – §19.7 we face up to the issues of inverse problems. Section 19.4 is an introduction to this large subject.

We should note here that wavelet transforms, already discussed in §13.10, are applicable not only to data compression and signal processing, but can also be used to transform some classes of integral equations into sparse linear problems that allow fast solution. You may wish to review §13.10 as part of reading this chapter.

Some subjects, such as *integro-differential equations*, we must simply declare to be beyond our scope. For a review of methods for integro-differential equations, see Brunner [4].

It should go without saying that this one short chapter can only barely touch on a few of the most basic methods involved in this complicated subject.

CITED REFERENCES AND FURTHER READING:

- Delves, L.M., and Mohamed, J.L. 1985, *Computational Methods for Integral Equations* (Cambridge, UK: Cambridge University Press).[1]
 Linz, P. 1985, *Analytical and Numerical Methods for Volterra Equations* (Philadelphia: S.I.A.M.).[2]
 Atkinson, K.E. 1976, *A Survey of Numerical Methods for the Solution of Fredholm Integral Equations of the Second Kind* (Philadelphia: S.I.A.M.).[3]
 Brunner, H. 1988, in *Numerical Analysis 1987*, Pitman Research Notes in Mathematics vol. 170, D.F. Griffiths and G.A. Watson, eds. (Harlow, Essex, UK: Longman Scientific and Technical), pp. 18–38.[4]
 Smithies, F. 1958, *Integral Equations* (Cambridge, UK: Cambridge University Press).
 Kanwal, R.P. 1971, *Linear Integral Equations* (New York: Academic Press).
 Green, C.D. 1969, *Integral Equation Methods* (New York: Barnes & Noble).

19.1 Fredholm Equations of the Second Kind

We desire a numerical solution for $f(t)$ in the equation

$$f(t) = \lambda \int_a^b K(t, s) f(s) ds + g(t) \quad (19.1.1)$$

The method we describe, a very basic one, is called the *Nystrom method*. It requires the choice of some approximate *quadrature rule*:

$$\int_a^b y(s) ds = \sum_{j=0}^{N-1} w_j y(s_j) \quad (19.1.2)$$

Here the set $\{w_j\}$ are the weights of the quadrature rule, while the N points $\{s_j\}$ are the abscissas.

What quadrature rule should we use? It is certainly possible to solve integral equations with low-order quadrature rules like the extended trapezoidal or Simpson's

rules. We will see, however, that the solution method involves $O(N^3)$ operations, and so the most efficient methods tend to use high-order quadrature rules to keep N as small as possible. For smooth, nonsingular problems, nothing beats Gaussian quadrature (e.g., Gauss-Legendre quadrature, §4.6). (For nonsmooth or singular kernels, see §19.3.)

Delves and Mohamed [1] investigated methods more complicated than the Nystrom method. For straightforward Fredholm equations of the second kind, they concluded “... the clear winner of this contest has been the Nystrom routine ... with the N -point Gauss-Legendre rule. This routine is extremely simple. ... Such results are enough to make a numerical analyst weep.”

If we apply the quadrature rule (19.1.2) to equation (19.1.1), we get

$$f(t) = \lambda \sum_{j=0}^{N-1} w_j K(t, s_j) f(s_j) + g(t) \quad (19.1.3)$$

Evaluate equation (19.1.3) at the quadrature points:

$$f(t_i) = \lambda \sum_{j=0}^{N-1} w_j K(t_i, s_j) f(s_j) + g(t_i) \quad (19.1.4)$$

Let f_i be the vector $f(t_i)$, g_i the vector $g(t_i)$, K_{ij} the matrix $K(t_i, s_j)$, and define

$$\tilde{K}_{ij} = K_{ij} w_j \quad (19.1.5)$$

Then, in matrix notation, equation (19.1.4) becomes

$$(\mathbf{1} - \lambda \tilde{\mathbf{K}}) \cdot \mathbf{f} = \mathbf{g} \quad (19.1.6)$$

This is a set of N linear algebraic equations in N unknowns that can be solved by standard triangular decomposition techniques (§2.3) — that is where the $O(N^3)$ operations count comes in. The solution is usually well-conditioned, unless λ is very close to an eigenvalue.

Having obtained the solution at the quadrature points $\{t_i\}$, how do you get the solution at some other point t ? You do *not* simply use polynomial interpolation. This destroys all the accuracy you have worked so hard to achieve. Nystrom’s key observation was that you should use equation (19.1.3) as an interpolatory formula, maintaining the accuracy of the solution.

Our routine for solving linear Fredholm equations of the second kind is coded as the object `Fred2`. The constructor sets up equation (19.1.6) and then solves it by LU decomposition with `LUdcmp`. The Gauss-Legendre quadrature is implemented by first getting the weights and abscissas with a call to `gauleg`. The routine `Fred2` requires that you provide an external function or functor that returns $g(t)$ and another that returns λK_{ij} . It then computes the solution f at the quadrature points in the member variable `f`. It also stores the quadrature points and weights. These are used by the member function `fredin` to carry out the Nystrom interpolation of equation (19.1.3) and return the value of f at any point in the interval $[a, b]$.

To be sure the usage is clear, here is the calling sequence when you have coded the external routines as functions:


```

Doub g(const Doub t) { ... }
Doub ak(const Doub t, const Doub s) { ... }
...
Fred2<Doub (Doub), Doub (Doub,Doub)> fred2(a,b,n,g,ak);
Doub ans=fred2.fredin(x);

```

If the external routines are functors Gfunc and Kernel, say, then the declarations are

```

Gfunc g;                      This could have arguments if you like.
Kernel ak;
Fred2<Gfunc, Kernel> fred2(a,b,n,g,ak);

```

Here is the routine:

```

template <class G, class K>                                         fred2.h
struct Fred2 {
Solves a linear Fredholm equation of the second kind.
    const Doub a,b;
    const Int n;
    G &g;
    K &ak;
    VecDoub t,f,w;
    Fred2(const Doub aa, const Doub bb, const Int nn, G &gg, K &akk) :
        a(aa), b(bb), n(nn), g(gg), ak(akk), t(n), f(n), w(n)
    Quantities a and b are input as the limits of integration. The quantity n is the number of
    points to use in the Gaussian quadrature. g and ak are user-supplied functions or functors
    that respectively return  $g(t)$  and  $\lambda K(t,s)$ . This constructor computes arrays t[0..n-1]
    and f[0..n-1] containing the abscissas  $t_i$  of the Gaussian quadrature and the solution  $f$ 
    at these abscissas. Also computed is the array w[0..n-1] of Gaussian weights for use with
    the Nystrom interpolation routine fredin.
    {
        MatDoub omk(n,n);
        gauleg(a,b,t,w);          Replace gauleg with another routine if not using
        for (Int i=0;i<n;i++) {    Gauss-Legendre quadrature.
            for (Int j=0;j<n;j++) Form  $\mathbf{1} - \lambda \tilde{\mathbf{K}}$ .
                omk[i][j]=Doub(i == j)-ak(t[i],t[j])*w[j];
            f[i]=g(t[i]);
        }
        LUdcmp alu(omk);          Solve linear equations.
        alu.solve(f,f);
    }

    Doub fredin(const Doub x)
    Given arrays t[0..n-1] and w[0..n-1] containing the abscissas and weights of the Gauss-
    ian quadrature, and given the solution array f[0..n-1], this function returns the value of
     $f$  at x using the Nystrom interpolation formula.
    {
        Doub sum=0.0;
        for (Int i=0;i<n;i++) sum += ak(x,t[i])*w[i]*f[i];
        return g(x)+sum;
    }
};

```

One disadvantage of a method based on Gaussian quadrature is that there is no simple way to obtain an estimate of the error in the result. The best practical method is to increase N by 50%, say, and treat the difference between the two estimates as a conservative estimate of the error in the result obtained with the larger value of N .

Turn now to solutions of the homogeneous equation. If we set $\lambda = 1/\sigma$ and $g = 0$, then equation (19.1.6) becomes a standard eigenvalue equation,

$$\tilde{\mathbf{K}} \cdot \mathbf{f} = \sigma \mathbf{f} \quad (19.1.7)$$

which we can solve with any convenient matrix eigenvalue routine (see Chapter 11). Note that if our original problem had a symmetric kernel, then the matrix $\mathbf{K}$ would be symmetric. However, since the weights w_j are not equal for most quadrature rules, the matrix $\tilde{\mathbf{K}}$ (equation 19.1.5) is not symmetric. The matrix eigenvalue problem is much easier for symmetric matrices, and so we should restore the symmetry if possible. Provided the weights are positive (which they are for Gaussian quadrature), we can define the diagonal matrix $\mathbf{D} = \text{diag}(w_j)$ and its square root, $\mathbf{D}^{1/2} = \text{diag}(\sqrt{w_j})$. Then equation (19.1.7) becomes

$$\mathbf{K} \cdot \mathbf{D} \cdot \mathbf{f} = \sigma \mathbf{f}$$

Multiplying by $\mathbf{D}^{1/2}$, we get

$$\left(\mathbf{D}^{1/2} \cdot \mathbf{K} \cdot \mathbf{D}^{1/2} \right) \cdot \mathbf{h} = \sigma \mathbf{h} \quad (19.1.8)$$

where $\mathbf{h} = \mathbf{D}^{1/2} \cdot \mathbf{f}$. Equation (19.1.8) is now in the form of a symmetric eigenvalue problem.

Solution of equations (19.1.7) or (19.1.8) will in general give N eigenvalues, where N is the number of quadrature points used. For square-integrable kernels, these will provide good approximations to the lowest N eigenvalues of the integral equation. Kernels of *finite rank* (also called *degenerate* or *separable* kernels) have only a finite number of nonzero eigenvalues (possibly none). You can diagnose this situation by a cluster of eigenvalues σ that are zero to machine precision. The number of nonzero eigenvalues will stay constant as you increase N to improve their accuracy. Some care is required here: A nondegenerate kernel can have an infinite number of eigenvalues that have an accumulation point at $\sigma = 0$. You distinguish the two cases by the behavior of the solution as you increase N . If you suspect a degenerate kernel, you will usually be able to solve the problem by analytic techniques described in all the textbooks.

CITED REFERENCES AND FURTHER READING:

- Delves, L.M., and Mohamed, J.L. 1985, *Computational Methods for Integral Equations* (Cambridge, UK: Cambridge University Press).[1]
 Atkinson, K.E. 1976, *A Survey of Numerical Methods for the Solution of Fredholm Integral Equations of the Second Kind* (Philadelphia: S.I.A.M.).

19.2 Volterra Equations

Let us now turn to Volterra equations, of which our prototype is the Volterra equation of the second kind,

$$f(t) = \int_a^t K(t, s) f(s) ds + g(t) \quad (19.2.1)$$

Most algorithms for Volterra equations march out from $t = a$, building up the solution as they go. In this sense they resemble not only forward substitution (as discussed in §19.0), but also initial value problems for ordinary differential equations. In fact, many algorithms for ODEs have counterparts for Volterra equations.

The simplest way to proceed is to solve the equation on a mesh with uniform spacing:

$$t_i = a + ih, \quad i = 0, 1, \dots, N, \quad h \equiv \frac{b-a}{N} \quad (19.2.2)$$

To do so, we must choose a quadrature rule. For a uniform mesh, the simplest scheme is the trapezoidal rule, equation (4.1.11):

$$\int_a^{t_i} K(t_i, s) f(s) ds = h \left(\frac{1}{2} K_{i0} f_0 + \sum_{j=1}^{i-1} K_{ij} f_j + \frac{1}{2} K_{ii} f_i \right) \quad (19.2.3)$$

Thus the trapezoidal method for equation (19.2.1) is

$$f_0 = g_0$$

$$(1 - \frac{1}{2} h K_{ii}) f_i = h \left(\frac{1}{2} K_{i0} f_0 + \sum_{j=1}^{i-1} K_{ij} f_j \right) + g_i, \quad i = 1, \dots, N \quad (19.2.4)$$

(For a Volterra equation of the first kind, the leading 1 on the left would be absent, and g would have opposite sign, with corresponding straightforward changes in the rest of the discussion.)

Equation (19.2.4) is an explicit prescription that gives the solution in $O(N^2)$ operations. Unlike Fredholm equations, it is not necessary to solve a system of linear equations. Volterra equations thus usually involve less work than the corresponding Fredholm equations, which, as we have seen, do involve the inversion of, sometimes large, linear systems.

The efficiency of solving Volterra equations is somewhat counterbalanced by the fact that *systems* of these equations occur more frequently in practice. If we interpret equation (19.2.1) as a *vector* equation for the vector of m functions $f(t)$, then the kernel $K(t, s)$ is an $m \times m$ matrix. Equation (19.2.4) must now also be understood as a vector equation. For each i , we have to solve the $m \times m$ set of linear algebraic equations by Gaussian elimination.

The routine `voltra` below implements this algorithm. You must supply an external function or functor that returns the k th function of the vector $g(t)$ at the point t and another that returns the (k, l) element of the matrix $K(t, s)$ at (t, s) . The routine `voltra` then returns the vector $f(t)$ at the regularly spaced points t_i .

```
template <class G, class K>
void voltra(const Doub t0, const Doub h, G &g, K &ak, VecDoub_O &t, MatDoub_O &f)
Solves a set of m linear Volterra equations of the second kind using the extended trapezoidal
rule. On input, t0 is the starting point of the integration and h is the stepsize. g(k,t) is
a user-supplied function or functor that returns  $g_k(t)$ , while ak(k,l,t,s) is another user-
supplied function or functor that returns the  $(k, l)$  element of the matrix  $K(t, s)$ . The solution
is returned in f[0..m-1][0..n-1], with the corresponding abscissas in t[0..n-1], where n-1
is the number of steps to be taken. The value of m is determined from the row-dimension of
the solution matrix f.
{
    Int m=f.nrows();
```

`voltra.h`

```

Int n=f.ncols();
VecDoub b(m);
MatDoub a(m,m);
t[0]=t0;
for (Int k=0;k<m;k++) f[k][0]=g(k,t[0]);           Initialize.
for (Int i=1;i<n;i++) {                               Take a step h.
    t[i]=t[i-1]+h;
    for (Int k=0;k<m;k++) {
        Doub sum=g(k,t[i]);                         Accumulate right-hand side of linear
        for (Int l=0;l<m;l++) {                       equations in sum.
            sum += 0.5*h*ak(k,l,t[i],t[0])*f[l][0];
            for (Int j=1;j<i;j++)
                sum += h*ak(k,l,t[i],t[j])*f[l][j];
            if (k == l)                               Left-hand side goes in matrix a.
                a[k][l]=1.0-0.5*h*ak(k,l,t[i],t[i]);
            else
                a[k][l] = -0.5*h*ak(k,l,t[i],t[i]);
        }
        b[k]=sum;
    }
    LUdcmp alu(a);                                   Solve linear equations.
    alu.solve(b,b);
    for (Int k=0;k<m;k++) f[k][i]=b[k];
}
}

```

For nonlinear Volterra equations, equation (19.2.4) holds with the product $K_{ii} f_i$ replaced by $K_{ii}(f_i)$, and similarly for the other two products of K 's and f 's. Thus, for each i we solve a nonlinear equation for f_i with a known right-hand side. Newton's method (§9.4 or §9.6) with an initial guess of f_{i-1} usually works very well provided the stepsize is not too big.

Higher-order methods for solving Volterra equations are, in our opinion, not as important as for Fredholm equations, since Volterra equations are relatively easy to solve. However, there is an extensive literature on the subject. Several difficulties arise. First, any method that achieves higher order by operating on several quadrature points simultaneously will need a special method to get started, when values at the first few points are not yet known.

Second, stable quadrature rules can give rise to unexpected instabilities in integral equations. For example, suppose we try to replace the trapezoidal rule in the algorithm above with Simpson's rule. Simpson's rule naturally integrates over an interval $2h$, so we easily get the function values at the even mesh points. For the odd mesh points, we could try appending one panel of the trapezoidal rule. But to which end of the integration should we append it? We could do one step of the trapezoidal rule followed by all Simpson's rule, or Simpson's rule with one step of the trapezoidal rule at the end. Surprisingly, the former scheme is unstable, while the latter is fine!

A simple approach that can be used with the trapezoidal method given above is Richardson extrapolation: Compute the solution with stepsizes h and $h/2$. Then, assuming the error scales with h^2 , compute

$$f_E = \frac{4f(h/2) - f(h)}{3} \quad (19.2.5)$$

This procedure can be repeated as with Romberg integration.

The general consensus is that the best of the higher-order methods is the *block-by-block method* (see [1]). Another important topic is the use of variable stepsize